Connecting Firebase &

Designing Your Database in FlutterFlow

A Complete Tutorial for FlutterFlow Developers

Written by:

Wahab Waris

Introduction

Almost every app needs a backend — somewhere to store users, posts, listings, ratings, orders, and more. Firebase is FlutterFlow's most deeply integrated backend, and getting it set up correctly from the start saves you from painful restructuring later.

This tutorial walks you through:

1. Creating a Firebase project
2. Connecting Firebase to FlutterFlow
3. Enabling Authentication
4. Designing a Firestore database structure (collections, documents, fields)
5. Creating Firestore collections inside FlutterFlow
6. Setting up Firestore Security Rules (basics)
7. Common mistakes and best practices

By the end, you'll have a properly connected Firebase backend with a clean, scalable database structure ready for your app.

1. Create a Firebase Project

Step 1: Go to Firebase Console

- Open https://console.firebase.google.com in your browser.
- Sign in with the Google account you want to use for this project.
- Click Add Project (or Create a project).

Step 2: Name Your Project

- Enter a project name (e.g., RateMyPet-App).
- Click Continue.
- Disable Google Analytics if you don't need it yet (optional — can be added later), or leave it enabled.
- Click Create Project, then wait for setup to finish, and click Continue.

2. Connect Firebase to FlutterFlow

Step 1: Open Firebase Settings in FlutterFlow

- In your FlutterFlow project, go to the left sidebar → Settings & Integrations.
- Click on Firebase.

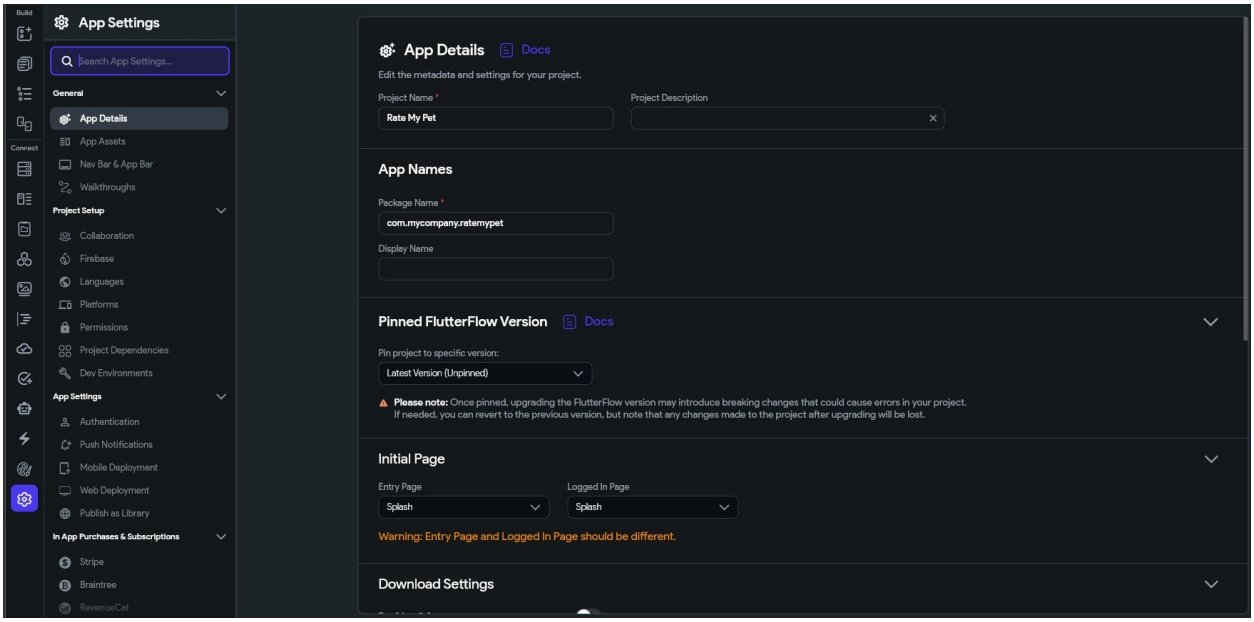


Figure 1: FlutterFlow App Settings — App Details page showing project name, package name, and initial page configuration

Step 2: Start the Connection

- Click Connect to an existing Firebase Project (or Create a new Firebase Project directly from FlutterFlow if you skipped Section 1).

- Sign in with the same Google account used in Firebase Console.
- Select the project you just created (e.g., RateMyPet-App) from the list.

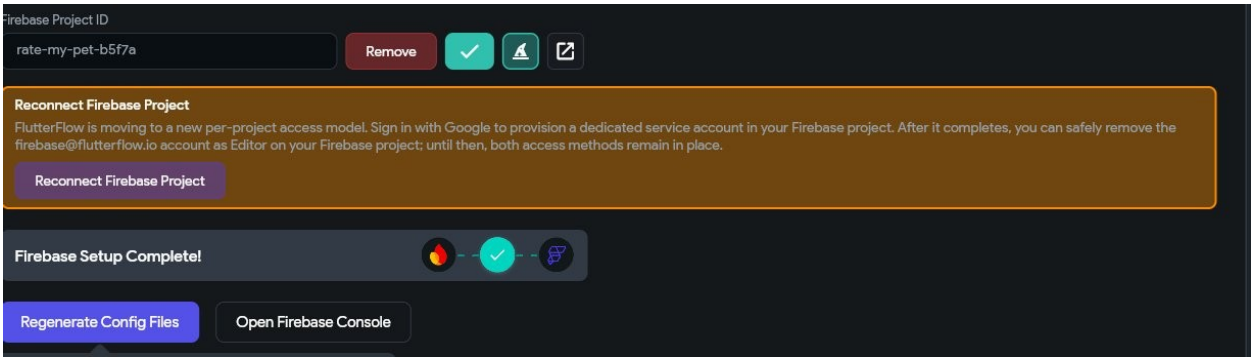


Figure 2: Firebase Project connection panel in FlutterFlow — showing the project ID field and 'Reconnect Firebase Project' prompt

Step 3: Configure Platforms

FlutterFlow will ask which platforms you're targeting (Android, iOS, Web).

- Check the platforms your app will run on.
- Enter your package name / bundle ID (e.g., com.yourcompany.ratemypet) — keep this consistent, you'll need it again for app store deployment.
- Click Confirm.

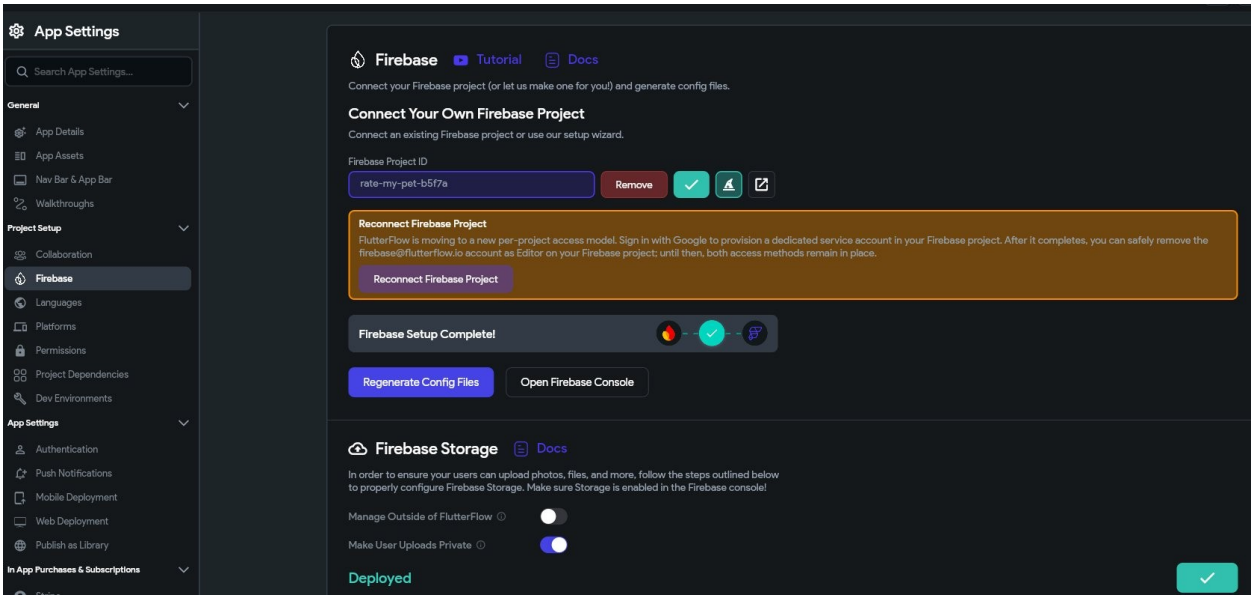


Figure 3: FlutterFlow App Settings — Firebase section showing 'Firebase Setup Complete!' status and Firebase Storage configuration with Make User Uploads Private enabled

Step 4: Wait for Sync

FlutterFlow will automatically:

- Create the Firebase config files
- Enable Firestore
- Set up Firebase Authentication (default settings)

- Link Firebase Storage

This usually takes 1–2 minutes. Once done, you'll see a green "Connected" status.

> 💡 After connecting, click 'Regenerate Config Files' if you make changes to your Firebase project (e.g., add new platforms or change the project ID).

---

3. Enable Authentication Methods

Step 1: Open Authentication Settings

- Still inside Settings & Integrations → Firebase, scroll to the Authentication section.
- Or go directly to Firebase Console → Build → Authentication → Sign-in method.

Step 2: Enable Sign-In Providers

Common options:

- Email/Password — most basic, good starting point
- Google Sign-In
- Apple Sign-In (required if you support iOS and offer any other social login)
- Phone Authentication

8. Toggle ON the providers you need.
9. For Google/Apple, you may need to provide additional config (OAuth client IDs, etc.) — FlutterFlow provides guided steps for each.

> ⚠️ Start with Email/Password during development — it's the fastest to test. Add social logins closer to launch since they require extra console configuration (OAuth credentials, SHA keys, etc.).

---

4. Designing Your Firestore Database Structure

This is the step most beginners rush — and regret later. Good structure now = fewer headaches later.

Core Concepts

- Collection → like a folder/table (e.g., users, listings, ratings)
- Document → like a single row/record inside a collection (e.g., one user)
- Field → like a column/property inside a document (e.g., name, email, avgRating)
- Sub-collection → a collection nested inside a specific document

Example: "Rate My Pet" App Structure

```
users (collection)
  └── {userId} (document)
       ├── name: string
       ├── email: string
       ├── role: string
       ├── followers_count: int
       ├── following_count: int
       └── created_at: timestamp

listings (collection)
  └── {listingId} (document)
       ├── title: string
       ├── description: string
       ├── owner_id: string (reference to users)
       ├── location: geopoint
       ├── avg_rating: double
       ├── total_ratings: int
       ├── photos: list<string>
       └── created_at: timestamp

ratings (collection)
  └── {ratingId} (document)
       ├── listing_id: string
       ├── user_id: string
       ├── rating_value: double
       ├── comment: string
       └── created_at: timestamp
```

Design Principles to Follow

10. Keep documents flat where possible — avoid deeply nested objects; Firestore queries work best on top-level fields.
11. Use references (document IDs) instead of duplicating large data — e.g., store owner_id, not the entire user object, inside a listing.
12. Denormalize for performance when needed — e.g., storing avg_rating and total_ratings directly on the listings document (synced via a Cloud Function) avoids expensive aggregation queries every time.
13. Plan your collections around your screens — ask: "What data does each screen need to query?" That usually reveals your collection structure naturally.
14. Use consistent naming — pick either camelCase or snake_case for field names and stick with it across the entire project.

5. Creating Firestore Collections in FlutterFlow

Step 1: Open Firestore Collections

- In the left sidebar, click on Firestore Collections (sometimes shown as a database icon).

Step 2: Create a New Collection

15. Click + Add Collection.
16. Enter the Collection Name (e.g., listings) — use lowercase, plural names for consistency.
17. Click Create.

Step 3: Add Fields to the Collection

18. Inside the collection, click + Add Field.
19. Enter the Field Name (e.g., title).

Choose the Field Type:

- String, Integer, Double, Boolean
- Timestamp, Geopoint
- Reference (link to another collection's document)
- List (for arrays — e.g., List<String> for photo URLs)
- DataType (for nested structured objects)

20. Repeat for every field in your design.

⚠️ Field types in FlutterFlow must match what your Cloud Functions / Custom Actions expect. If a Cloud Function writes a double but FlutterFlow expects an int, you'll get type-mismatch errors at runtime.

Step 4: Repeat for All Collections

Create each collection from your design (users, listings, ratings, etc.) following the same process.

---

6. Firestore Security Rules — The Basics

Security rules control who can read/write what. By default, FlutterFlow sets reasonably safe rules, but you should understand and review them.

Step 1: Open Rules in Firebase Console

- Go to Firebase Console → Build → Firestore Database → Rules tab.

Step 2: Understand a Basic Rule Example

```
rules_version = '2';
service cloud.firestore {
  match /databases/{database}/documents {
    // Users can read any profile, but only edit their own
    match /users/{userId} {
      allow read: if true;
      allow write: if request.auth != null && request.auth.uid == userId;
```

```
    }
    // Anyone signed in can read listings; only the owner can edit/delete
    match /listings/{listingId} {
      allow read: if true;
      allow create: if request.auth != null;
      allow update, delete: if request.auth != null &&
                            request.auth.uid == resource.data.owner_id;
    }
    // Ratings: signed-in users can create, but not edit others' ratings
    match /ratings/{ratingId} {
      allow read: if true;
      allow create: if request.auth != null;
      allow update, delete: if request.auth != null &&
                            request.auth.uid == resource.data.user_id;
    }
  }
}
```

Step 3: Test & Publish

21. Use the Rules Playground (in Firebase Console) to simulate reads/writes before publishing.
22. Click Publish once you're confident the rules are correct.

⚠️ Never leave rules wide open (allow read, write: if true;) in a production app — this allows anyone to read/write/delete your entire database.

7. Common Mistakes & Best Practices

Mistake	Why It's a Problem	Best Practice
Designing collections randomly as you build	Leads to inconsistent structure, broken queries later	Sketch your data model before creating collections
Storing entire objects instead of references	Bloats documents, causes sync issues when data changes	Use Reference fields + Cloud Functions to sync summary data
Leaving security rules at allow read, write: if true	Major security risk — anyone can access/delete data	Restrict rules based on request.auth and ownership
Mixing field naming styles (avgRating vs avg_rating)	Causes confusion and bugs in Custom Actions/Cloud Functions	Pick one naming convention and use it everywhere
Not testing Authentication providers in development	Login breaks in production unexpectedly	Test each provider (Email, Google, Apple) before launch
Forgetting to add indexes for compound queries	Queries fail with "missing index" errors	Firestore will give a direct link to create the needed index — click

Mistake	Why It's a Problem	Best Practice
		it

Conclusion

A well-connected Firebase project and a thoughtfully designed Firestore database are the foundation of every successful FlutterFlow app. Spend the extra hour planning your collections and security rules upfront — it will save you days of rework later, especially once Cloud Functions, Custom Actions, and multiple developers start depending on that structure.

This tutorial is part of our internal training resource series. For questions or suggestions, reach out to the author.

Author: Wahab Waris